Développement de logiciels cryptographiques

Bibliothèques de calcul

Nicolas Aragon

Problématique

En cryptographie, on doit manipuler des objets mathématiques complexes :

- RSA → entiers de **plusieurs milliers de bits**
- Log discret → **arithmétique modulaire** (Z/nZ, courbes elliptiques)
- Crypto à base de **codes** → calculs dans des **corps finis**

Problématique

- C / C++ → arithmétique **basique** (simple/double précision)
- Python, Sage, Magma → arithmétique **multi-précision** (entiers “illimités”), objets mathématiques complexes

> Pourquoi ne pas rester en Python ?

- **Performance** (lenteur, systèmes embarqués, etc.)
- **Sécurité** (contrôle fin pour limiter les canaux auxiliaires)

La bibliothèque GMP

- **GNU Multiple Precision** arithmetic library
- Écrite en **C**
- Types principaux :
  - `mpz_t` — entiers ($\mathbb{Z}$)
  - `mpq_t` — rationnels ($\mathbb{Q}$)
  - `mpf_t` — flottants ($\mathbb{R}$, voir aussi MPFR)

👉 Exemple d'illustration : **factorielle** sans débordement.

Factorielle en C

Sans GMP : utilisation d'une variable de type `unsigned int`.

⚠️ Dépassement dès `13!` (4 octets → max 2^32 - 1)

Avec GMP : aucun dépassement !

✅ `2000!`, `100000!` → calculables très rapidement

Compiler avec GMP

Inclure l'en-tête :

```
#include "gmp.h"
```

Compilation :

```
gcc mon_programme.c -lgmp -o mon_programme
```

> Variante en deux étapes : `gcc -c ...` puis `gcc ... -lgmp -o ...`

Structure d'un `mpz_t`

Un grand entier dans GMP contient :

- un **tableau de mots** contenant les données,
- la **taille allouée**,
- la **taille utilisée**.

⚠️ Toujours **initialiser / libérer** :

- `mpz_init(mpz_t x)` … `mpz_clear(mpz_t x)`
- versions variadiques : `mpz_inits(...)`, `mpz_clears(...)`

Affectations & conversions

Pour affecter une valeur à un grand entier on utilise la fonction `mpz_set_XXX`, où `XXX` peut être :

- Rien, pour affecter un autre `mpz_t`,
- `ui` pour un entier non signé,
- `si` pour un entier signé,
- `d` pour un double,
- `q` pour un rationnel,
- `f` pour un flottant,
- `str` pour une chaine.

Affectations & conversions

Les conversions fonctionnent de la même manière !

Quelques exemples :

- `mpz_get_ui(z)`, `mpz_get_si(z)`, `mpz_get_d(z)`
- `mpz_get_str(NULL, base, z)`

⚠️ Vous ne pouvez pas mettre un grand entier, dans un type simple ne pouvant pas le contenir !

I/O formatées :

- `gmp_printf("%Zd %Zx\n", z_dec, z_hex)` (pas `printf` standard)

Fonctions arithmétiques

- Addition / soustraction / produit : `mpz_add`, `mpz_sub`, `mpz_mul`
- Exponentiation :
  - **non modulaire** : `mpz_pow_ui(mpz_t rop, mpz_t base, long int exp)`
  - **modulaire** : `mpz_powm(mpz_t rop, mpz_t base, mpz_t exp, mpz_t mod)`

Fonctions arithmétiques

Quelques autres fonctions utiles pour la cryptographie basée sur la factorisation :

- PGCD & inverse :
  - `mpz_gcd(g, a, b)`, `mpz_gcdext(g, s, t, a, b)`
  - `mpz_invert(inv, a, mod)` si `gcd(a,mod)=1`
- Primalité & premiers :
  - `mpz_probab_prime_p(n, reps)`
  - `mpz_nextprime(p, n)`

Comparaisons & opérations bit à bit

Comparaisons :

- `mpz_cmp(a, b)`, `mpz_cmp_ui(a, u)`, `mpz_cmpabs(a, b)`

Bitwise :

- `mpz_and`, `mpz_ior`, `mpz_xor`, `mpz_com`
- Tests/manips de bits : `mpz_tstbit`, `mpz_setbit`, `mpz_clrbit`

Entrées/sorties

Comme on l'a vu, la fonction `gmp_printf` permet d'écrire sur la sortie standard, avec des modificateurs spécifiques.

De manière plus générale on utilisera :

- `mpz_out_str` pour les sorties,
- `mpz_inp_str` pour les entrées.

Génération pseudo-aléatoire

GMP implémente le type `gmp_randstate_t` pour la génération aléatoire. Il doit être initialisé / libéré, et peut être "seedé".

Tirages :

- `mpz_urandomb(x, state, nbits)` → `x ∈ [0, 2^nbits[`
- `mpz_urandomm(x, state, bound)` → `x ∈ [0, bound[`

```
gmp_randstate_t prng;
gmp_randinit_default(prng);
gmp_randseed_ui(prng, time(NULL)); // pas pour la crypto !
mpz_t a, bound; mpz_inits(a, bound, NULL);
mpz_set_ui(bound, 10000);
mpz_urandomb(a, prng, 20);
```

Génération pseudo-aléatoire

GMP utilise une fonction mathématique itérée afin de générer des nombres pseudo-aléatoires. Vous avez essentiellement le choix entre deux fonctions :

- Un "Mersenne Twister", qui est l'option par défaut,
- Un générateur à congruence linéaire.

⚠️ Pour de vraies applications cryptographiques, aucune de ces deux méthodes n'est suffisante. On pourrait, par exemple, itérer l'exécution d'une fonction de hachage sécurisée.

Démo — Exponentiation modulaire

On veut calculer :

$$r = b^e \bmod n$$

- Naïf : **square-and-multiply** (binaire)
- GMP : `mpz_powm`
- Variante **“sec”** : accès mémoire / instructions constantes (contre les canaux auxiliaires)

> L'implémentation de GMP va deux fois plus vite sur des entiers de 50000 bits !

Démo — Factorielle

- Naïf : Le temps d'exécution augmente rapidement.
- GMP : `mpz_fac`
  - **1 000 000!** en **≈ 0,12 s**

→ Les fonctions implémentées par GMP embarquent des optimisations non triviales !

Interface et classes C++

Le C est parfois fastidieux :

- Allocation et désallocation manuelle,
- Pas de **surcharge d'opérateurs**.

Afin de simplifier l'écriture GMP propose **une interface et des classes C++**.

Interface et classes C++

Les classes sont les suivantes :

- `mpz_class` pour les entiers,
- `mpq_class` pour les rationnels,
- `mpf_class` pour les flottants.

> Plus d'allocation manuelle, et des opérateurs surchargés !

Interface et classes C++

Exemple d'utilisation :

```
mpz_class a, b, c;
a = 1234;
b = "-5678";
c = a + b;
```

> Les générateurs aléatoires ont aussi leur propre classe.

Interface et classes C++

Pour des cas spécifiques on peut passer de la classe au type "de base" de GMP et vice-versa.

- `a.get_mpz_t()`
- `mpz_class a(z)`

La bibliothèque NTL

Dans d'autres contextes (par exemple, la **cryptographie post-quantique**) on veut effectuer des calculs, non pas sur de grands entiers, mais dans **des corps finis**.

> Dans ce cours, nous allons nous concentrer sur des extensions de $\mathbb{F}_2$

Corps finis

Rappel : un **corps** est un ensemble d'éléments dans lequel on peut réaliser les opérations suivantes :

- Addition / Soustraction
- Multiplication
- **Division** (tout élément non nul admet un inverse multiplicatif)

> Le terme "fini" indique que le corps ne contient pas un nombre infini d'éléments.

Extensions de $\mathbb{F}_2$

Une extension de $\mathbb{F}_2$ de degré n est un corps qui contient $\mathbb{F}_2^n$ éléments.

Nous allons le définir comme $\mathbb{F}_2[X]/P$, où P est un polynome irréductible de degré n.

> Exemple : le corps $\mathbb{F}_2^8$ utilisé dans AES est défini comme :
>
> $$\mathbb{F}_2[X]/(X^8 + X^4 + X^3 + X + 1)$$

Extensions de $\mathbb{F}_2$

Concrètement :

- Un élément du corps est un polynome à coefficients dans $\mathbb{F}_2$, de degré au plus $n-1$
- L'**addition** et la **soustraction** sont celles des polynomes
- La **multiplication** est la multiplication de polynomes modulo P
- La **division** par A est la multiplication par l'inverse de A modulo P

La bibliothèque NTL

NTL (A Library for doing Number Thoery) est une bibliothèque implémentant de nombreux objets mathématiques en C++. Ceux qui nous intéressent sont les suivants :

- **GF2E**, les extensions de $\mathbb{F}_2$
- **vec_GF2E**, les vecteurs d'éléments de $\mathbb{F}_2^n$
- **mat_GF2E**, les matrices d'éléments de $\mathbb{F}_2^n$

Exemple : construction et utilisation du corps de l'AES